

/\*

### Queue Interface:

- > The Queue interface of the Java collections framework provides the functionality of the queue data structure.
- > It extends the Collection interface.
- > Since the Queue is an interface, we cannot provide the direct implementation of it.
- > In order to use the functionalities of Queue, we need to use classes that implement it:
  1. LinkedList
  2. PriorityQueue
  3. ArrayDeque
- > The Queue interface is also extended by various subinterfaces:
  1. Deque
  2. BlockingQueue
  3. BlockingDeque
- > In queues, elements are stored and accessed in First In, First Out manner.
- > That is, elements are added from the behind and removed from the front.

### NOTES:

- It was introduced in JDK 5.0 version.
- It is a direct child interface to Collection Interface.
- It able to arrange all the elements as per FIFO [First In First Out], but, it is possible to change this algorithm as per our requirement.
- It able to allow duplicate elements.
- It is not following Insertion order.
- It is following Sorting order.
- It will not allow null values, if we add null value then JVM will rise an Exception like `java.lang.NullPointerException`.
- It will not allow heterogeneous elements, if we add heterogeneous elements then JVM will rise an exception like `java.lang.ClassCastException`.
- It able to allow only homogeneous elements.
- It able to allow comparable objects by default, if we add non comparable objects then JVM will rise an exception like `java.lang.ClassCastException`.
- If we want to add non comparable objects then we have to use Comparator.
- It able to manage all elements prior to process.

### --> Methods of Queue:

- > The Queue interface includes all the methods of the Collection interface.
- > It is because Collection is the super interface of Queue.
- > Some of the commonly used methods of the Queue interface are:
  1. `add()` - Inserts the specified element into the queue. If the task is successful, `add()` returns true, if not it throws an exception.
  2. `offer()` - Inserts the specified element into the queue. If the task is successful, `offer()` returns true, if not it returns false.
  3. `element()` - Returns the head of the queue. Throws an exception if the queue is empty.
  4. `peek()` - Returns the head of the queue. Returns null if the queue is empty.
  5. `remove()` - Returns and removes the head of the queue. Throws an exception if the queue is empty.
  6. `poll()` - Returns and removes the head of the queue. Returns null if the queue is empty.

-----> PriorityQueue <-----

- It was introduced in JDK 5.0 version.
- It is not Legacy Collection.
- It is a direct implementation class to Queue interface.
- It able to arrange all the elements prior to processing on the basis of the priorities.
- It able to allow duplicate elements.
- It is not following Insertion order.
- It is following Sorting order.
- It will not allow null values, if we add null value then JVM will rise an Exception like `java.lang.NullPointerException`.
- It will not allow heterogeneous elements, if we add heterogeneous elements then JVM will rise an exception like `java.lang.ClassCastException`.
- It able to allow only homogeneous elements.
- It able to allow comparable objects by default, if we add non comparable objects then JVM will rise an exception like `java.lang.ClassCastException`.
- If we want to add non comparable objects then we have to use Comparator.
- Its initial capacity 11 elements.
- It is not synchronized .
- No method is synchronized in PriorityQueue.
- It allows more than one thread at a time to access data.
- It follows parallel execution.
- It able to reduce application execution time.
- It able to improve application performance.
- It is not giving guarantee for Data consistency.
- It is not threadsafe.

#### PriorityQueue Constructors:

1. `public PriorityQueue()` - creates empty PriorityQueue Object

Ex: `PriorityQueue p = new PriorityQueue();`

2. `public PriorityQueue(int capacity)` - creates empty queue with specified capacity

3. `public PriorityQueue (Collection c)` - created PriorityQueue object with all the elements of the specified collection.

4. `public PriorityQueue (Comparator c)`

5. `public PriorityQueue (int capacity, Comparator c)`

\*/

**import** java.util.\*;

```
public class Collection7 {
    public static void main(String[] args) {

        // Implementing PriorityQueue Class

        ArrayList<Integer> al = new ArrayList<>();
        al.add(30);
        al.add(20);
        al.add(10);
        al.add(40);
        System.out.println("ArrayList is: "+al);
    }
}
```

```

Queue q1 = new PriorityQueue<>(Comparator.reverseOrder());
System.out.println(q1);
System.out.println(q1.size());

PriorityQueue<Integer> pq1 = new PriorityQueue<>(a1);
System.out.println(pq1);
System.out.println(pq1.size());

PriorityQueue<Integer> pq2 = new PriorityQueue<>(10);

// 1. public boolean offer (Object obj) - insert specified element to Queue.

q1.offer(12);
q1.offer(2);
q1.offer(10);
// q1.offer("14"); // R.E: Class Cast Exception
q1.offer(9);
q1.offer(10);
q1.offer(8);
q1.offer(18);
// q1.offer(null); // R.E: NullPointerException

System.out.println(q1);
System.out.println();

// 2. public Object peek() - It can be used to return head element of the Queue.

System.out.println("Use of peek() method: " + q1.peek());

// 3. public Object element() - It can be used to return head element of the Queue.

System.out.println("Use of element() method: " + q1.element());

/*
NOTE:
--> If we access peek() method on an empty Queue then peek() will return
    "null" value.
--> If we access element() method on an empty Queue then element() method
    will rise an exception like java.util.NoSuchElementException.
*/

// System.out.println(pq2.peek()); --> null
// System.out.println(pq2.element()); --> R.E.: NoSuchElementException

// 4. public Object poll() - used to return and remove head element from Queue.

System.out.println(q1.poll());
System.out.println(q1);

// 5. public Object remove() - used to return and remove head element from Queue.

System.out.println(q1.remove());
System.out.println(q1);

/*
NOTE:
--> If we access poll() method on an empty Queue then poll() method will
    return "null" value.
--> If we access remove() method on an empty Queue then remove() method will

```

```
        rise an exception like "java.util.NoSuchElementException".
    */

    // System.out.println(pq2.poll()); --> null
    // System.out.println(pq2.remove()); --> R.E: java.util.NoSuchElementException

    // 6. Iterator iterator()

    Iterator itr = q1.iterator();
    while (itr.hasNext()) {
        System.out.println(itr.next());
    }

}

}
```

/\*

Deque interface:

--> The Deque interface of the Java collections framework provides the functionality of a double-ended queue.

--> It extends the Queue interface.

--> In a regular queue, elements are added from the rear and removed from the front.

--> However, in a deque, we can insert and remove elements from both front and rear.

--> In order to use the functionalities of the Deque interface, we need to use classes that implement it:

1. LinkedList
2. ArrayDeque

--> Since Deque extends the Queue interface, it inherits all the methods of the Queue interface.

Besides methods available in the Queue interface, the Deque interface also includes the following methods:

1. void addFirst() - Adds the specified element at the beginning of the deque. Throws an exception if the deque is full.

2. void addLast() - Adds the specified element at the end of the deque. Throws an exception if the deque is full.

3. boolean offerFirst() - Adds the specified element at the beginning of the deque. Returns false if the deque is full.

4. boolean offerLast() - Adds the specified element at the end of the deque. Returns false if the deque is full.

5. Object getFirst() - Returns the first element of the deque. Throws an exception if the deque is empty.

6. Object getLast() - Returns the last element of the deque. Throws an exception if the deque is empty.

7. Object peekFirst() - Returns the first element of the deque. Returns null if the deque is empty.

8. Object peekLast() - Returns the last element of the deque. Returns null if the deque is empty.

9. Object removeFirst() - Returns and removes the first element of the deque. Throws an exception if the deque is empty.

10. Object removeLast() - Returns and removes the last element of the deque. Throws an exception if the deque is empty.

11. Object pollFirst() - Returns and removes the first element of the deque. Returns null if the deque is empty.

12. Object pollLast() - Returns and removes the last element of the deque. Returns null if the deque is empty.

13. boolean add() - Adds elements

14. boolean addAll() - Adds all the elements of given objects

15. void clear() - removes all the elements

16. boolean isEmpty() - returns boolean if given object is empty

17. Object remove() - removes first element from given object

18. boolean remove(Object o) - removes specific element from given object, if given element not available then returns false.

18. int size() - gives count of total number of elements present in the object

--> The Stack class of the Java Collections framework provides the implementation of the stack.

--> However, it is recommended to use Deque as a stack instead of the Stack class.

--> It is because methods of Stack are synchronized.

Here are the methods the Deque interface provides to implement stack:

1. `push()` - adds an element at the beginning of deque
2. `pop()` - removes an element from the beginning of deque
3. `peek()` - returns an element from the beginning of deque

\*/

```
import java.util.ArrayDeque;
import java.util.ArrayList;
import java.util.Deque;
```

```
public class Collection8 {
```

```
    public static void main(String[] args) {
```

```
        ArrayList<Integer> al = new ArrayList<>();
        al.add(30);
        al.add(20);
        al.add(10);
        al.add(40);
        System.out.println("ArrayList is: "+al);
```

```
        ArrayDeque<Integer> dq = new ArrayDeque<>();
        ArrayDeque dq1 = new ArrayDeque<>(10);
        ArrayDeque dq2 = new ArrayDeque<>(al);
```

```
        System.out.println("dq is: "+dq);
        System.out.println("dq1 is: "+dq1);
        System.out.println("dq2 is: " + dq2);
```

```
        dq1.add(12);
        dq1.add("Hi");
```

```
        dq.addAll(dq2);
```

```
        dq1.offer(13);
        dq1.offer("Hey");
        dq1.offerFirst(10); // NullPointerException if element passed is "null"
        dq1.offerLast(20.32); // NullPointerException if element passed is "null"
```

```
        dq.addFirst(9); // NullPointerException if element passed is "null"
        dq.addLast(16); // NullPointerException if element passed is "null"
```

```
        System.out.println(dq1);
```

```
        System.out.println(dq1.getFirst()); // NoSuchElementException if empty
        System.out.println(dq1.getLast()); // NoSuchElementException if empty
```

```
        System.out.println(dq1.peek()); // peek returns null if empty
```

```
        System.out.println(dq1.peekFirst()); // peekFirst returns null if empty
        System.out.println(dq1.peekLast()); // peekLast returns null if empty
```

```
        System.out.println(dq1.remove());
        System.out.println(dq1.remove(10));
```

```
        System.out.println(dq1.removeFirst()); // NoSuchElementException if empty
```

```
System.out.println(dq1.removeLast()); // NoSuchElementException if empty
```

```
System.out.println(dq1.poll()); // poll returns null if empty
```

```
System.out.println(dq1.pollFirst()); // pollFirst returns null if empty
```

```
System.out.println(dq1.pollLast()); // pollLast returns null if empty
```

```
dq.clear();
```

```
System.out.println(dq1.size());
```

```
}
```

```
}
```

/\*

--> Set interface:

- It was introduced in JDK 1.2 version.
- It is a direct child interface to Collection interface.
- It is not index based, it able to arrange all the elements on the basis of elements hashcode values.
- It will not allow duplicate elements.
- It will not follow insertion order.

Note: LinkedHashSet will follow insertion order.

- It will not follow Sorting order.

Note: SortedSet, NavigableSet and TreeSet are following Sorting order.

- It able to allow only one null value.

Note: SortedSet, NavigableSet and TreeSet are not allowing even single null value.

--> HashSet:

- HashSet is a direct implementation class to Set interface.
- It was introduced in JDK 1.2 version.
- It is not index based, it able to arrange all the elements on the basis of elements hashcode values.
- It will not allow duplicate elements.
- It will not follow insertion order.
- It will not follow Sorting order.
- It able to allow only one null value.
- Its internal data structure is "Hashtable".
- Its initial capacity is "16" elements and its initial fill\_Ratio is 75%.
- It is not synchronized.
- Almost all the methods are not synchronized in HashSet
- It allows more than one thread at a time.
- It follows parallel execution.
- It will reduce execution time.
- It improves performance of the applications.
- It is not giving guarantee for data consistency.
- It is not threadsafe.

--> Constructors:

1. public HashSet():

--> This constructor can be used to create an empty HashSet object with 16 elements as initial capacity and 75% fill ratio.

EX: HashSet hs = new HashSet();

2. public HashSet(int capacity):

--> This constructor can be used to create an empty HashSet object with the specified capacity as initial capacity and with the default fill ratio 75%.

EX: HashSet hs = new HashSet(20);

3. public HashSet(int capacity, float fill\_Ratio):

--> This constructor can be used to create an empty HashSet object with the specified capacity and with the specified fill ratio.

EX: HashSet hs = new HashSet(20,0.85f);

4. `public HashSet(Collection c)`

--> This constructor can be used to create HashSet object with all the elements of the specified Collection.

`*/`

```
import java.util.ArrayList;
import java.util.HashSet;
import java.util.Iterator;
```

```
public class Collection9 {
    public static void main(String[] args) {
```

*// Constructors:*

*// 1. boolean add (Element e):*

```
ArrayList al1 = new ArrayList<>();
al1.add(10);
al1.add(20);
al1.add(20);
al1.add(10);
al1.add(30);
```

```
HashSet hs1 = new HashSet<>();
System.out.println("10 added ? "+hs1.add(10));
hs1.add(20);
hs1.add(30);
System.out.println("10 added again? "+hs1.add(10));
hs1.add(40);
hs1.add(50);
```

```
System.out.println(hs1);
```

```
HashSet hs2 = new HashSet<>(10, 0.75f);
```

```
HashSet hs3 = new HashSet<>(5);
```

```
HashSet hs4 = new HashSet<>(al1);
System.out.println(hs4);
```

*// 2. boolean remove(Object o):*

```
System.out.println(hs1.remove(10));
System.out.println(hs1.remove(100));
```

```
System.out.println(hs1);
```

*// 3. void clear():*

```
System.out.println(hs1.isEmpty());
System.out.println(hs2.isEmpty());
```

*// 4. boolean addAll (Collection c):*

```
hs2.addAll(al1);
```

```
System.out.println(hs2);
```

*// 5. boolean removeAll (Collection c):*

```
hs1.removeAll(al1);
System.out.println(hs1);

// 6. boolean equals (Object o):

System.out.println("a1 is: " + al1);
System.out.println("hs2 is: "+hs2);
System.out.println("hs4 is: "+hs4);

System.out.println(hs2.equals(al1));
System.out.println(hs2.equals(hs4));

// 7. int size():

System.out.println(hs1.size());
System.out.println(hs2.size());

// 8. Iterator iterator():

System.out.println("Use of Iterator: ");

Iterator itr = hs2.iterator();
while (itr.hasNext()) {
    System.out.println(itr.next());
}
}
```

/\*

*--> Map interface:*

- It was introduced in JDK 1.2 version.
- It is not child interface to Collection Interface.
- It able to arrange all the elements in the form of Key-value pairs.
- In Map, both keys and values are objects.
- Duplicates are not allowed at keys, but values may be duplicated.
- Only one null value is allowed at keys side, but, any number of null values are allowed at values side.
- Both keys and Values are able to allow heterogeneous elements.
- Insertion order is not followed.
- Sorting order is not followed.

*--> HashMap:*

- It was introduced in JDK1.2 version.
- It is not Legacy.
- It is an implementation class to Map interface.
- It able to arrange all the elements in the form of Key-value pairs.
- In HashMap, both keys and values are objects.
- Duplicates are not allowed at keys, but values may be duplicated.
- Only one null value is allowed at keys side, but, any number of null values are allowed at values side.
- Both keys and Values are able to allow heterogeneous elements.
- Insertion order is not followed.
- Sorting order is not followed.
- Its internal data structure is "Hashtable".
- Its initial capacity is 16 elements.
- It is not synchronized.
- No method is synchronized in HashMap.
- It allows more than one thread to access data.
- It follows parallel execution.
- It will reduce application execution time.
- It will improve application performance.
- It is not giving guarantee for data consistency.
- It is not threadsafe.

*--> Constructors:*

1. *public HashMap()*
2. *public HashMap(int capacity)*
3. *public HashMap(int capacity, float fill\_Ratio)*
4. *public HashMap(Map m)*

*\* Methods:*

1. *public void put(Object key, Object value):*  
*--> It will add the specified key-value pair to Map.*
2. *public void putAll(Map m):*  
*--> It will add all key-value pairs of the specified map to the present Map object.*
3. *public Object get(Object key):*  
*--> It will return value of the specified key.*
4. *public Object remove(Object key):*  
*--> It will remove a key-value pair from Map on the basis of the specified key.*
5. *public int size():*

--> It will return number of key-value pairs of a Map.

6. `public boolean containsKey(Object key):`

--> It will check whether the specified key is existed or not at keys side.

7. `public boolean cotainsValue(Object key):`

--> It will check whether the specified value is available or not at values side.

8. `public Set keySet():`

--> It will return all keys in the form of a Set.

9. `public Collection values():`

--> It will return all values in the form of a Collection object.

10. `public boolean isEmpty():`

--> It will check whether the Map object is empty or not.

--> If the present map object is empty then it will return true value otherwise it will return false value.

`*/`

```
import java.util.Collection;
```

```
import java.util.HashMap;
```

```
import java.util.Set;
```

```
public class Collection10 {
```

```
    public static void main(String[] args) {
```

```
        HashMap<Integer,String> hm1 = new HashMap<>();
```

```
        HashMap hm2 = new HashMap<>(10);
```

```
        HashMap hm3 = new HashMap<>(10, 0.75f);
```

```
        // 1. public void put(Object key, Object value):
```

```
        hm1.put(1, "CVM");
```

```
        hm1.put(2, "HDS");
```

```
        hm1.put(3, "PAT");
```

```
        //hm1.put(1, "MHM");
```

```
        hm1.put(null, null);
```

```
        hm1.put(4, null);
```

```
        hm1.put(0, "PAT");
```

```
        hm1.put(-9, "VYP");
```

```
        hm1.put(-8, "VYP");
```

```
        System.out.println(hm1);
```

```
        HashMap hm4 = new HashMap<>(hm1);
```

```
        System.out.println(hm4);
```

```
        // 2. public void putAll(Map m):
```

```
        hm2.putAll(hm3);
```

```
        System.out.println(hm2);
```

```
        // 3. public Object get(Object key):
```

```
        System.out.println(hm1.get("1"));
```

```
// 4. public Object remove(Object key):  
//     public Object remove (Object key, Object value):  
  
System.out.println(hm1.remove(1, "ABC"));  
System.out.println(hm1.remove(2));  
  
System.out.println(hm1);  
  
// 5. public int size():  
  
System.out.println(hm1.size());  
  
// 6. public boolean containsKey(Object key):  
  
System.out.println(hm1.containsKey(1));  
System.out.println(hm1.containsKey(2));  
  
// 7. public boolean containsValue(Object key):  
  
System.out.println(hm1.containsValue("CVM"));  
System.out.println(hm1.containsValue("ABC"));  
  
// 8. public Set keySet():  
  
Set s = hm1.keySet();  
System.out.println(s);  
  
// 9. public Collection values():  
  
Collection c = hm1.values();  
System.out.println(c);  
  
// 10. public boolean isEmpty():  
  
System.out.println(hm1.isEmpty());  
System.out.println(hm3.isEmpty());
```

```
}
```

```
}
```

```
/*
--> Java Hashtable class:

• Java Hashtable class implements a hashtable, which maps keys to values.
• It inherits Dictionary class and implements the Map interface.
```

NOTE:

- A Hashtable is an array of a list. Each list is known as a bucket.
- The position of the bucket is identified by calling the hashCode() method.
- A Hashtable contains values based on the key.
- Java Hashtable class contains unique elements.
- Java Hashtable class doesn't allow null key or value.
- Java Hashtable class is synchronized.
- The initial default capacity of Hashtable class is 11.

\* Constructors:

1. Hashtable ()
2. Hashtable (int initialcapacity)
3. Hashtable (int initialcapacity, float loadfactor)
4. .Hashtable (Map t)

```
*/
```

```
import java.util.*;
import java.util.Collection;
import java.util.Hashtable;
import java.util.Map;
import java.util.Set;
```

```
public class Collection11 {
    public static void main(String[] args) {

        Hashtable<String, Integer> ht1 = new Hashtable();
        Hashtable<String, Integer> ht2 = new Hashtable(15);
        Hashtable<String, Integer> ht3 = new Hashtable(15, 0.60f);

        // 1. public void put(Object key, Object value):
        //     public Object putIfAbsent(Object key, Object Value):

        ht1.put("CVM", 123);
        ht1.put("VYP", 789);
        ht1.put("PAT", 234);
        ht1.put("HDS", 345);
        ht1.put("MHM", 456);
        ht1.put("CVM", 12);
        System.out.println(ht1.putIfAbsent("CVM", 121));
        System.out.println(ht1.putIfAbsent("ABC", 121));

        System.out.println("ht1 is: "+ht1);

        // 2. public void putAll(Map m):

        Hashtable<String, Integer> ht4 = new Hashtable(ht1);
        ht4.putAll(ht1);
        System.out.println(ht4);

        // 3. public Object get(Object key):
```

```

System.out.println(ht1.get("CVM"));

// 4. public Object remove(Object key):
//     public Object remove (Object key, Object value):

System.out.println(ht1.remove("MHM", 456));
System.out.println(ht1.remove("CVM"));

System.out.println(ht1);

// 5. public int size():

System.out.println(ht1.size());

// 6. public boolean containsKey(Object key):

System.out.println(ht1.containsKey("PAT"));
System.out.println(ht1.containsKey("MHM"));

// 7. public boolean containsValue(Object key):

System.out.println(ht1.containsValue(234));
System.out.println(ht1.containsValue(123));

// 8. public Set keySet():

Set s = ht1.keySet();
System.out.println(s);

// 9. public Collection values():

Collection c = ht1.values();
System.out.println(c);

// 10. public boolean isEmpty():

System.out.println(ht1.isEmpty());
System.out.println(ht2.isEmpty());

// 11. Map.Entry:

for (Map.Entry m : ht1.entrySet()) {
    System.out.println(m.getKey() + " = " + m.getValue());
}
}

```

```
/*
```

Title: Student Record Management System

Problem Statement:

You are tasked with developing a Student Record Management System in Java that allows users to manage student records.

The system should utilize the File, BufferedReader, BufferedWriter, Hashtable, and ArrayDeque classes to perform the necessary operations efficiently.

All inputs and outputs should be handled using the Scanner class.

The system should have the following functionalities:

Add a Student Record:

Users should be able to add a new student record to the system. Each record will have a unique roll number, name, and marks.

Remove a Student Record:

Users should be able to remove an existing student record from the system by specifying the roll number.

Search for a Student Record:

Users should be able to search for a student record by providing the roll number. The system should display the student's name and marks if found.

Display All Student Records:

Users should be able to view the complete student record, displaying all saved records.

Requirements:

The student records should be persisted in a text file named "student\_records.txt".

The system should use a Hashtable to store the student records, with the roll number as the key and a custom object representing the student information as the value.

The custom object should include attributes for roll number, name, and marks.

The File class should be used to read from and write to the "student\_records.txt" file.

The BufferedReader class should be used to read from the file.

The BufferedWriter class should be used to write to the file.

All inputs should be taken using the Scanner class to interact with the user.

```
*/
```

```
import java.io.*;
```

```
import java.util.*;
```

```
class Student {
```

```
    private String rollNumber;
```

```
    private String name;
```

```
    private double marks;
```

```
    public Student(String rollNumber, String name, double marks) {
```

```
        this.rollNumber = rollNumber;
```

```
        this.name = name;
```

```
        this.marks = marks;
```

```
    }
```

```
    public String getRollNumber() {
```

```
        return rollNumber;
```

```
    }
```

```
    public String getName() {
```

```

        return name;
    }

    public double getMarks() {
        return marks;
    }

    @Override
    public String toString() {
        return "Roll Number: " + rollNumber + ", Name: " + name + ", Marks: " + marks;
    }
}

class StudentRecordManagementSystem {
    private Hashtable<String, Student> studentRecords;
    private static final String FILE_NAME = "student_records.txt";

    public StudentRecordManagementSystem() {
        studentRecords = new Hashtable<>();
    }

    public void addStudentRecord(String rollNumber, String name, double marks) {
        Student newStudent = new Student(rollNumber, name, marks);
        studentRecords.put(rollNumber, newStudent);
    }

    public void removeStudentRecord(String rollNumber) {
        if (studentRecords.containsKey(rollNumber)) {
            studentRecords.remove(rollNumber);
        }
    }

    public void searchStudentRecord(String rollNumber) {
        Student student = studentRecords.get(rollNumber);
        if (student != null) {
            System.out.println("Student found:");
            System.out.println(student);
        } else {
            System.out.println("Student not found.");
        }
    }

    public void displayAllStudentRecords() {
        if (studentRecords.isEmpty()) {
            System.out.println("No student records found.");
            return;
        }

        System.out.println("Student Records:");
        for (String rollNumber : studentRecords.keySet()) {
            System.out.println(studentRecords.get(rollNumber));
        }
    }

    public void saveStudentRecordsToFile() {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(FILE_NAME))) {
            for (Student student : studentRecords.values()) {

```

```

        writer.write(student.getRollNumber() + "," + student.getName() + "," +
student.getMarks());
        writer.newLine();
    }
} catch (IOException e) {
    e.printStackTrace();
}
}

public void loadStudentRecordsFromFile() {
    try (BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME))) {
        String line;
        while ((line = reader.readLine()) != null) {
            String[] parts = line.split(",");
            if (parts.length == 3) {
                String rollNumber = parts[0];
                String name = parts[1];
                double marks = Double.parseDouble(parts[2]);
                Student student = new Student(rollNumber, name, marks);
                studentRecords.put(rollNumber, student);
            }
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

public static void main(String[] args) {
    StudentRecordManagementSystem recordManagementSystem = new
StudentRecordManagementSystem();
    recordManagementSystem.loadStudentRecordsFromFile();

    Scanner scanner = new Scanner(System.in);
    int choice;
    String rollNumber, name;
    double marks;

    do {
        System.out.println("\nStudent Record Management System");
        System.out.println("1. Add a Student Record");
        System.out.println("2. Remove a Student Record");
        System.out.println("3. Search for a Student Record");
        System.out.println("4. Display All Student Records");
        System.out.println("5. Exit");
        System.out.print("Enter your choice: ");
        choice = scanner.nextInt();
        scanner.nextLine(); // Clear the newline character from the buffer

        switch (choice) {
            case 1:
                System.out.print("Enter Roll Number: ");
                rollNumber = scanner.nextLine();
                System.out.print("Enter Name: ");
                name = scanner.nextLine();
                System.out.print("Enter Marks: ");
                marks = scanner.nextDouble();
                recordManagementSystem.addStudentRecord(rollNumber, name, marks);
                break;

            case 2:

```

```
        System.out.print("Enter Roll Number of the student to remove: ");
        rollNumber = scanner.nextLine();
        recordManagementSystem.removeStudentRecord(rollNumber);
        break;

    case 3:
        System.out.print("Enter Roll Number of the student to search: ");
        rollNumber = scanner.nextLine();
        recordManagementSystem.searchStudentRecord(rollNumber);
        break;

    case 4:
        recordManagementSystem.displayAllStudentRecords();
        break;

    case 5:
        System.out.println("Exiting Student Record Management System.");
        break;

    default:
        System.out.println("Invalid choice. Please try again.");
        break;
    }
} while (choice != 5);

recordManagementSystem.saveStudentRecordsToFile();
scanner.close();
}
```

```
}
```